

```

INSTALL redis-sentinel
CC redis-cli.o
LINK redis-cli
CC redis-benchmark.o
LINK redis-benchmark
INSTALL redis-check-rdb
CC redis-check-aof.o
LINK redis-check-aof

```

hint: It's a good idea to run 'make test' ;)

Figure 1: Redis installation

```

[root@example redis-3.2.8]# ls
00-RELEASENOTES  CONTRIBUTING  docs  Makefile  README.md  redis-sentinel  src  utils
[root@example redis-3.2.8]# make
[root@example redis-3.2.8]#

```

Figure 2: Redis configuration file

If the above error occurs, then use the command given below:

```
make MALLOC=libc && make install
```

Figure 1 shows that Redis has been successfully installed.

Starting Redis

Just take a look at the directory *redis-3.2.8* (Figure 2). You will find the *redis.conf* file in it. In order to start Redis, we need to configure this file.

Let us consider a main configuration of the file:

```
bind 127.0.0.1
```

If you are using a static IP, then you can give the static IP address; otherwise, 127.0.0.1 is okay.

The default port of Redis is 6379.

```
pidfile /var/run/redis_6379.pid
```

The above line gives the location of *pidfile*, and if you are not running as root then you can give your custom path:

```
logfile ""
```

By default, it is blank but you can give the log file path; try to give the absolute path.

```
Like /var/run/redis_6379.log
```

Now, give the following commands:

```
logfile "/var/run/redis_6379.log"
dbfilename dump.rdb
```

The above line gives the name of the dump file that stores the database.

```
dir ./
```

The above option specifies the location of the *dump.rdb* file.

```
daemonize yes
```

By default, the above option is set as 'no'. When you run the Redis server, it shows progress on the terminal but, in a development environment, the Redis server must be run in the background. So, set it as 'yes'.

Let us now start Redis. In Figure 2, there is a directory *src*, which contains Redis binaries.

The command to run the Redis server is as follows:

```
# src/redis-server redis.conf
```

See Figure 3, which shows that Redis is running.

Redis supports master-slave replication, which allows slave Redis servers to be exact copies of master servers. The master/slave replication is one way of achieving scalability. The slave is *read-only* and the master is *read and write*. When a data update happens on the master, it eventually affects the slave.

```

[root@example redis-3.2.8]# src/redis-server redis.conf
[root@example redis-3.2.8]# ps -ef | grep redis
root      3745      1  0 15:14 7      00:00:00 src/redis-server 127.0.0.1:6379
root      3749  3591  0 15:14 pts/0      00:00:00 grep redis

```

Figure 3: Redis server in the running state

Creating a Redis slave

In order to start a slave, you will need a *conf* file. So make a copy of *redis.conf* as *redis-slave.conf*:

```
# cp redis.conf redis-slave.conf
```

Next, open *redis-slave.conf*. And change the settings as shown below:

```

pidfile /var/run/redis_6380.pid
logfile "/var/run/redis_6379-s.log"
port 6380
slaveof 127.0.0.1 6379

```

The above settings set the port number of the slave and tell the slave about its master.

Start the slave and follow the process, as shown in Figure 4. The Redis master and slave are both running now.

Redis commands

To run commands on the Redis remote server, we will use the *redis-cli* command syntax:

```
# redis-cli -h <ip_address> -p <port number>
```